



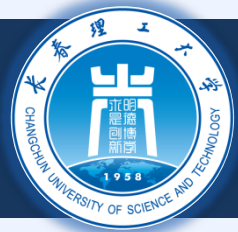
长春理工大学
Changchun University of Science and Technology

面向对象程序设计

8. 运算符重载

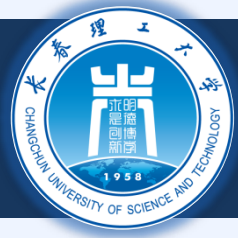
2022-03-25





提纲

- 8.1 引言
- 8.2 运算符重载的基础
- 8.3 运算符重载的限制
- 8.4 运算符重载的全局函数和成员函数比较
- 8.5 重载流插入运算符和流提取运算符
- 8.6 重载一元运算符
- 8.7 重载二元运算符
- 8.8 实例: Array 类
- 8.9 类型转换
- 8.10 实例: String 类
- 8.11 重载 ++ 和 --
- 8.12 实例: Date 类
- 8.13 标准库中的 string 和 vector 类



8.1 引言

◆ 在类对象上使用运算符(运算符重载)

- ✦ 对于某些操作来说，比函数调用要简洁、明了
- ✦ 是运算符符合所在的上下文环境

◆ 例

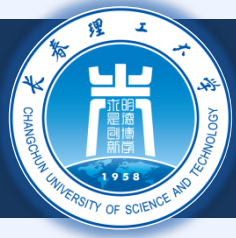
✦ <<

- ✦ 流提取符，位左移运算符

✦ +

- ✦ 应用与多种类型的数值计算 (integers, floats, etc.)

◆ 讨论什么时候需要操作符重载



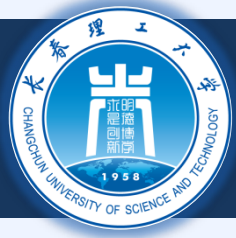
8.2 运算符重载的基础

◆ 类型

- ✦ 基本类型或者用户自定义类型
- ✦ 为用户自定义类型定义已有的运算符的新语义
 - ✦ 不可以创造新的运算符

◆ 重载运算符

- ✦ 为类创建一个**新函数**
- ✦ 函数名格式是： `operator` 关键字后接运算符
 - ✦ `Operator +` （重载加法运算符的函数名）



8.2 运算符重载的基础

◆ 在类对象上使用运算符

- ✦ 该运算符必须重载，也有例外：

- ✦ 赋值运算符： =

- ➡ 默认：对象的数据成员逐个赋值

- ✦ 取地址运算符： &

- ➡ 返回对象的地址

- ✦ 他们也可以被重载

◆ 重载提供的简洁的表达

- ✦ `object2 = object1.add(object2);`

- ✦ `object2 = object2 + object1;`



8.3 运算符重载的限制

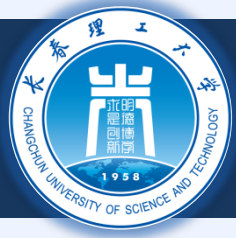
❖ 不可以改变:

- ✦ 已有的基本数据类型的操作语义
 - ✦ 比如, 不可以重新定义整数加法的语义
- ✦ 用算优先级 (计算的顺序)
 - ✦ 使用括号强制调整计算顺序
- ✦ 结合律 (left-to-right or right-to-left)
- ✦ 操作数的个数
 - ✦ `&` 是单目运算符, 只接受一个操作数

❖ 不能创造新的运算符

❖ 运算符必须显式地重载

- ✦ 重载 `+`, 与 `+=` 的语义无关



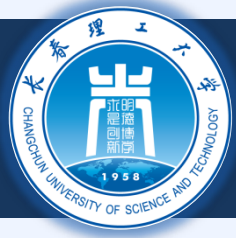
8.3 运算符重载的限制

Operators that can be overloaded

+	-	*	/	%	^	&	
~	!	=	<	>	+=	-=	*=
/=	%=	^=	&=	=	<<	>>	>>=
<<=	==	!=	<=	>=	&&		++
--	->*	,	->	[]	()	new	delete
new[]	delete[]						

Operators that cannot be overloaded

.	.*	::	?:	sizeof
---	----	----	----	--------



8.4 运算符重载的全局函数和成员函数比较

运算符函数

成员函数

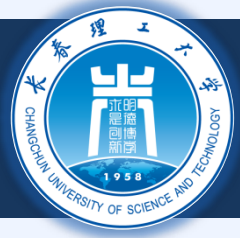
- ✦ 用 `this` 隐式获得实参，获得二元运算符的左操作数 (比如 `+`)
- ✦ 最左边的操作数必须是运算符的一个类对象

非成员函数

- ✦ 需要通过实参提供所有操作数
- ✦ 可以是来自不同类的对象
- ✦ 必须是友元函数才能访问私有和保护性成员

运算符成员函数在以下情况下被自动调用

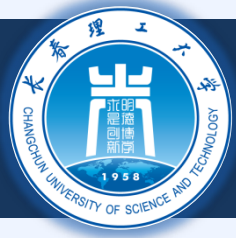
- ✦ 二元运算符的左操作数的确是该类的对象
- ✦ 一元运算符的单操作数的确是该类的对象



8.4 运算符重载的全局函数和成员函数比较

◆ 重载 << 运算符

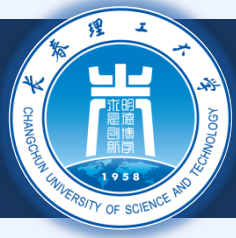
- ✦ 做操作数是 ostream &
 - ✦ 例如 cout << classObject 中的cout对象
- ✦ 类似， >> 的左操作数是 istream &
- ✦ 这两个必须是非成员函数



8.4 运算符重载的全局函数和成员函数比较

◆ 可交换的运算符

- ✦ 想要 + 可交换
 - ✦ “a + b” 和 “b + a” 都工作
- ✦ 假设我们有两个类
- ✦ 只有类对象在左操作数位置时，才能以类成员函数的形式进行运算符重载
 - ✦ HugelntClass + Long int
 - ✦ 可以是成员函数
- ✦ 否则需要非成员函数的形式实现
 - ✦ Long int + HugelntClass



8.5 重载流插入运算符和流提取运算符

◆ << 和 >>

- ✦ 对于基本类型来说，已经是重载过了
- ✦ 也可以处理用户自定义类型

◆ 例

- ✦ PhoneNumber 类
 - ✦ 保存一个电话号码
- ✦ 自动以规定形式输出电话号码
 - ✦ (123) 456-7890



8.5 重载流插入运算符和流提取运算符

```
2 // Overloading the stream-insertion and
3 // stream-extraction operators.
4 #include <iostream>
5
6 using std::cout;
7 using std::cin;
8 using std::endl;
9 using std::ostream;
10 using std::istream;
11
12 #include <iomanip>
13
14 using std::setw;
15
16 // PhoneNumber class definition
17 class PhoneNumber {
18     friend ostream &operator<<( ostream&, const PhoneNumber & );
19     friend istream &operator>>( istream&, PhoneNumber & );
20
21 private:
22     char areaCode[ 4 ]; // 3-digit area code and null
23     char exchange[ 4 ]; // 3-digit exchange and null
24     char line[ 5 ];     // 4-digit line and null
25
26 }; // end class PhoneNumber
```



8.5 重载流插入运算符和流提取运算符

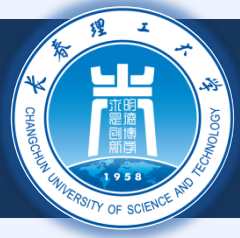
```
28 // overloaded stream-insertion operator; cannot be
29 // a member function if we would like to invoke it with
30 // cout << somePhoneNumber;
31 ostream &operator<<( ostream &output, const PhoneNumber &num )
32 {
33     output << "(" << num.areaCode << ")" "
34         << num.exchange << "-" << num.line;
35
36     return output; // enables cout << a << b << c;
37
38 } // end function operator<<
39
40 // overloaded stream-extraction operator; cannot be
41 // a member function if we would like to invoke it with
42 // cin >> somePhoneNumber;
43 istream &operator>>( istream &input, PhoneNumber &num )
44 {
45     input.ignore(); // skip (
46     input >> setw( 4 ) >> num.areaCode; // input area code
47     input.ignore( 2 ); // skip ) and space
48     input >> setw( 4 ) >> num.exchange; // input exchange
49     input.ignore(); // skip dash (-)
50     input >> setw( 5 ) >> num.line; // input line
51
52     return input; // enables cin >> a >> b >> c;
```



8.5 重载流插入运算符和流提取运算符

```
54     } // end function operator>>
55
56     int main()
57     {
58         PhoneNumber phone; // create object phone
59
60         cout << "Enter phone number in the form (123) 456-7890:\n";
61
62         // cin >> phone invokes operator>> by implicitly issuing
63         // the non-member function call operator>>( cin, phone )
64         cin >> phone;
65
66         cout << "The phone number entered was: " ;
67
68         // cout << phone invokes operator<< by implicitly issuing
69         // the non-member function call operator<<( cout, phone )
70         cout << phone << endl;
71
72         return 0;
73
74     } // end main
```

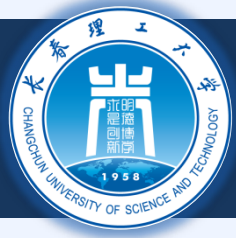
```
Enter phone number in the form (123) 456-7890:
(800) 555-1212
The phone number entered was: (800) 555-1212
```



8.6 重载一元运算符

◆ 重载一元运算符

- ✦ 非静态成员函数，无实参
- ✦ 非成员函数，1 个实参
 - ✦ 实参必须是类对象或者是类对象的引用
- ✦ 记住：静态成员函数仅仅能访问静态成员



8.6 重载一元运算符

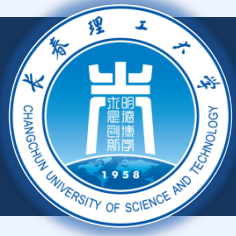
◆ 后面的一个例子 (8.10)

- ✦ 重载运算符 “!” 检查空字符串
- ✦ 如果是非静态成员函数，不需要实参
 - ✦ !s 等同于成员函数调用：s.operator!()

```
class String {  
    public:  
        bool operator!() const;  
    ...  
};
```

- ✦ 如果是非成员函数，需要一个实参
 - ✦ s! 等同于函数调用 operator!(s)

```
class String {  
    friend bool operator!( const String & )  
    ...  
};
```

8.7 重载二元运算符

◆ 重载二元运算符

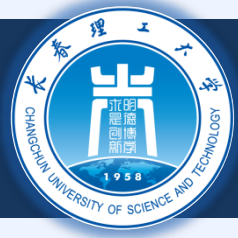
- ✦ 非静态成员函数，1 个实参
- ✦ 非成员函数，2 个实参
 - ✦ 一个实参必须是类对象或者是类对象引用

◆ 后面的例子

- ✦ 非静态成员函数，需要 1 个实参

```
class String {  
public:  
    const String &operator+=( const String & );  
    ...  
};
```

- ✦ $y += z$ 等同于 $y.operator+=(z)$



8.7 重载二元运算符

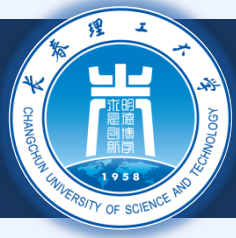
◆ 后面的例子：

✚ 非成员函数，需要 2 个实参

✚ 例：

```
class String {  
    friend const String &operator+=(  
        String &, const String & );  
    ...  
};
```

✚ $y += z$ 等同于函数调用 `operator+=( y, z )`



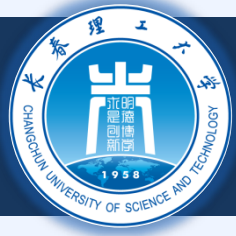
8.8 实例：Array 类

◆ C++ 中的数组

- 没有边界检查（访问越界）
- 不能进行有意义的比较，比如 `==`, `<=`, `>=`, `!=`
- 不能复制（数组名是一个指针常量）
- 不能整体输入输出，一次处理一个元素

◆ 例：实现一个 Array 类，能够：

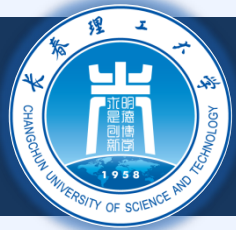
- 边界检查
- 赋值
- 知道自己的大小
- 整体输入输出 `<<` 和 `>>`
- 比较是否相等 `==` 和 `!=`



8.8 实例：Array 类

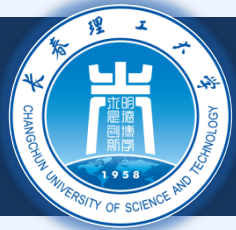
◆ 拷贝构造函数

- ✦ 当一个对象被拷贝的时候
 - ✦ 用一个已有的对象去初始化另外一个对象
 - ✦ `Array newArray( oldArray );`
- ✦ Array类拷贝构造函数的原型
 - ✦ `Array( const Array & );`
 - ✦ 参数必须是引用
 - ✦ 否则，当调用拷贝构造函数的时候会触发新的拷贝
 - ✦ 死循环



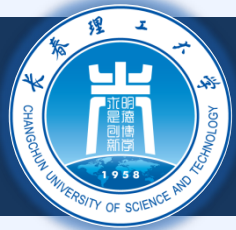
8.8 实例：Array 类

```
1 // array1.h
2 // Array class for storing arrays of integers.
3 #ifndef ARRAY1_H
4 #define ARRAY1_H
5
6 #include <iostream>
7
8 using std::ostream;
9 using std::istream;
10
11 class Array {
12     friend ostream &operator<<( ostream &, const Array & );
13     friend istream &operator>>( istream &, Array & );
14
15 public:
16     Array( int = 10 );           // default constructor
17     Array( const Array & );     // copy constructor
18     ~Array();                   // destructor
19     int getSize() const;        // return size
20
21     // assignment operator
22     const Array &operator=( const Array & );
23
24     // equality operator
25     bool operator==( const Array & ) const;
```



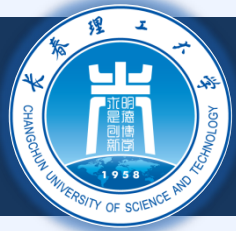
8.8 实例：Array 类

```
27 // inequality operator; returns opposite of == operator
28 bool operator!=( const Array &right ) const
29 {
30     return ! ( *this == right ); // invokes Array::operator==
31
32 } // end function operator!=
33
34 // subscript operator for non-const objects returns lvalue
35 int &operator[]( int );
36
37 // subscript operator for const objects returns rvalue
38 const int &operator[]( int ) const;
39
40 private:
41     int size; // array size
42     int *ptr; // pointer to first element of array
43
44 }; // end class Array
45
46 #endif
```



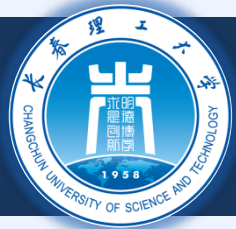
8.8 实例：Array 类

```
1 // array1.cpp
2 // Member function definitions for class Array
3 #include <iostream>
4
5 using std::cout;
6 using std::cin;
7 using std::endl;
8
9 #include <iomanip>
10
11 using std::setw;
12
13 #include <new> // C++ standard "new" operator
14
15 #include <cstdlib> // exit function prototype
16
17 #include "array1.h" // Array class definition
18
19 // default constructor for class Array (default size 10)
20 Array::Array( int arraySize )
21 {
22     // validate arraySize
23     size = ( arraySize > 0 ? arraySize : 10 );
24
25     ptr = new int[ size ]; // create space for array
```



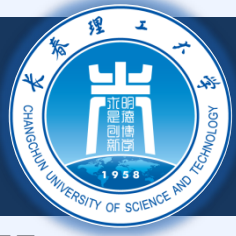
8.8 实例：Array 类

```
27     for ( int i = 0; i < size; i++ )
28         ptr[ i ] = 0;                // initialize array
29
30 } // end Array default constructor
31
32 // copy constructor for class Array;
33 // must receive a reference to prevent infinite recursion
34 Array::Array( const Array &arrayToCopy )
35     : size( arrayToCopy.size )
36 {
37     ptr = new int[ size ]; // create space for array
38
39     for ( int i = 0; i < size; i++ )
40         ptr[ i ] = arrayToCopy.ptr[ i ]; // copy into object
41
42 } // end Array copy constructor
43
44 // destructor for class Array
45 Array::~~Array()
46 {
47     delete [] ptr; // reclaim array space
48
49 } // end destructor
```

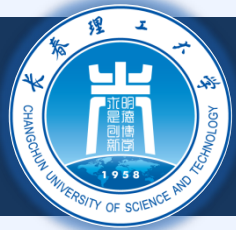
8.8 实例：Array 类

```
51 // return size of array
52 int Array::getSize() const
53 {
54     return size;
55
56 } // end function getSize
57 // overloaded assignment operator;
58 // const return avoids: ( a1 = a2 ) = a3
59 const Array &Array::operator=( const Array &right )
60 {
61     if ( &right != this ) { // check for self-assignment
62
63         // for arrays of different sizes, deallocate original
64         // left-side array, then allocate new left-side array
65         if ( size != right.size ) {
66             delete [] ptr; // reclaim space
67             size = right.size; // resize this object
68             ptr = new int[ size ]; // create space for array copy
69
70         } // end inner if
71
72         for ( int i = 0; i < size; i++ )
73             ptr[ i ] = right.ptr[ i ]; // copy array into object
74
75     } // end outer if
76 }
```



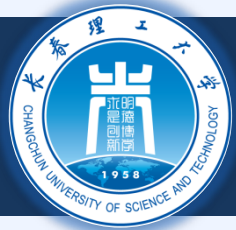
8.8 实例：Array 类

```
77
78     return *this;    // enables x = y = z, for example
79
80 } // end function operator=
81
82 // determine if two arrays are equal and
83 // return true, otherwise return false
84 bool Array::operator==( const Array &right ) const
85 {
86     if ( size != right.size )
87         return false;    // arrays of different sizes
88
89     for ( int i = 0; i < size; i++ )
90
91         if ( ptr[ i ] != right.ptr[ i ] )
92             return false; // arrays are not equal
93
94     return true;        // arrays are equal
95
96 } // end function operator==
97
```



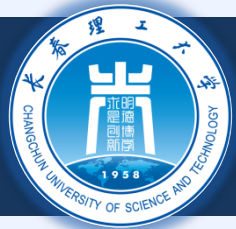
8.8 实例：Array 类

```
98 // overloaded subscript operator for non-const Arrays
99 // reference return creates an lvalue
100 int &Array::operator[]( int subscript )
101 {
102     // check for subscript out of range error
103     if ( subscript < 0 || subscript >= size ) {
104         cout << "\nError: Subscript " << subscript
105             << " out of range" << endl;
106
107         exit( 1 ); // terminate program; subscript out of range
108
109     } // end if
110
111     return ptr[ subscript ]; // reference return
112
113 } // end function operator[]
114
```



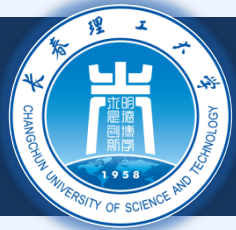
8.8 实例：Array 类

```
115 // overloaded subscript operator for const Arrays
116 // const reference return creates an rvalue
117 const int &Array::operator[]( int subscript ) const
118 {
119     // check for subscript out of range error
120     if ( subscript < 0 || subscript >= size ) {
121         cout << "\nError: Subscript " << subscript
122             << " out of range" << endl;
123         exit( 1 ); // terminate program; subscript out of range
124     } // end if
125
126     return ptr[ subscript ]; // const reference return
127 } // end function operator[]
128
129 // overloaded input operator for class Array;
130 // inputs values for entire array
131 istream &operator>>( istream &input, Array &a )
132 {
133     for ( int i = 0; i < a.size; i++ )
134         input >> a.ptr[ i ];
135
136     return input; // enables cin >> x >> y;
137 } // end function
```



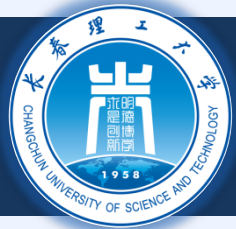
8.8 实例：Array 类

```
142
143 // overloaded output operator for class Array
144 ostream &operator<<( ostream &output, const Array &a )
145 {
146     int i;
147
148     // output private ptr-based array
149     for ( i = 0; i < a.size; i++ ) {
150         output << setw( 12 ) << a.ptr[ i ];
151
152         if ( ( i + 1 ) % 4 == 0 ) // 4 numbers per row of output
153             output << endl;
154
155     } // end for
156
157     if ( i % 4 != 0 ) // end last line of output
158         output << endl;
159
160     return output; // enables cout << x << y;
161
162 } // end function operator<<
```



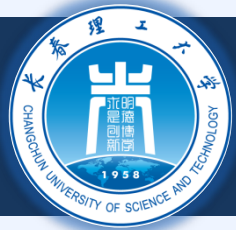
8.8 实例：Array 类

```
2 // Array class test program.
3 #include <iostream>
4
5 using std::cout;
6 using std::cin;
7 using std::endl;
8
9 #include "array1.h"
10
11 int main()
12 {
13     Array integers1( 7 ); // seven-element Array
14     Array integers2;      // 10-element Array by default
15
16     // print integers1 size and contents
17     cout << "Size of array integers1 is "
18         << integers1.getSize()
19         << "\nArray after initialization:\n" << integers1;
20
21     // print integers2 size and contents
22     cout << "\nSize of array integers2 is "
23         << integers2.getSize()
24         << "\nArray after initialization:\n" << integers2;
```



8.8 实例：Array 类

```
26 // input and print integers1 and integers2
27 cout << "\nInput 17 integers:\n";
28 cin >> integers1 >> integers2;
29
30 cout << "\nAfter input, the arrays contain:\n"
31      << "integers1:\n" << integers1
32      << "integers2:\n" << integers2;
33
34 // use overloaded inequality (!=) operator
35 cout << "\nEvaluating: integers1 != integers2\n";
36
37 if ( integers1 != integers2 )
38     cout << "integers1 and integers2 are not equal\n";
39
40 // create array integers3 using integers1 as an
41 // initializer; print size and contents
42 Array integers3( integers1 ); // calls copy constructor
43
44 cout << "\nSize of array integers3 is "
45      << integers3.getSize()
46      << "\nArray after initialization:\n" << integers3;
47
```



8.8 实例：Array 类

```
48 // use overloaded assignment (=) operator
49 cout << "\nAssigning integers2 to integers1:\n";
50 integers1 = integers2; // note target is smaller
51
52 cout << "integers1:\n" << integers1
53      << "integers2:\n" << integers2;
54
55 // use overloaded equality (==) operator
56 cout << "\nEvaluating: integers1 == integers2\n";
57
58 if ( integers1 == integers2 )
59     cout << "integers1 and integers2 are equal\n";
60
61 // use overloaded subscript operator to create rvalue
62 cout << "\nintegers1[5] is " << integers1[ 5 ];
63
64 // use overloaded subscript operator to create lvalue
65 cout << "\n\nAssigning 1000 to integers1[5]\n";
66 integers1[ 5 ] = 1000;
67 cout << "integers1:\n" << integers1;
68 // attempt to use out-of-range subscript
69 cout << "\nAttempt to assign 1000 to integers1[15]" << endl;
70 integers1[ 15 ] = 1000; // ERROR: out of range
71 return 0;
73
75 } // end main
```




8.8 实例：Array 类

Size of array integers1 is 7

Array after initialization:

0	0	0	0
0	0	0	

Size of array integers2 is 10

Array after initialization:

0	0	0	0
0	0	0	0
0	0		

Input 17 integers:

1 2 3 4 5 6 7 8 9 10 11 12 13 14 15 16 17

After input, the arrays contain:

integers1:

1	2	3	4
5	6	7	

integers2:

8	9	10	11
12	13	14	15



8.8 实例：Array 类

```
Evaluating: integers1 != integers2  
integers1 and integers2 are not equal
```

```
Size of array integers3 is 7
```

```
Array after initialization:
```

1	2	3	4
5	6	7	

```
Assigning integers2 to integers1:
```

```
integers1:
```

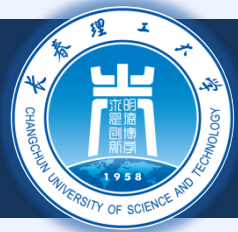
8	9	10	11
12	13	14	15
16	17		

```
integers2:
```

8	9	10	11
12	13	14	15
16	17		

```
Evaluating: integers1 == integers2  
integers1 and integers2 are equal
```

```
integers1[5] is 13
```



8.8 实例：Array 类

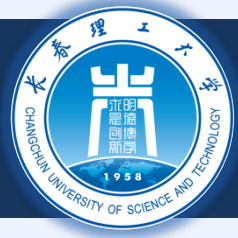
Assigning 1000 to integers1[5]

integers1:

8	9	10	11
12	1000	14	15
16	17		

Attempt to assign 1000 to integers1[15]

Error: Subscript 15 out of range



8.9 类型转换

◆ 类型转换

- ✦ 传统上是指，整型到浮点型之类的转换
- ✦ 用户自定义类型也需要转换

◆ 类型转换运算符 (转换运算符)

- ✦ 实现以下的转换
 - ✦ 一个类到另外一个类
 - ✦ 一个类到基本类型 (例如: int, char等)
- ✦ 必须是非静态成员函数
 - ✦ 不能是友元
- ✦ 不能指定返回类型
 - ✦ 返回类型其实就是对象正要转换成的目标类型



8.9 类型转换

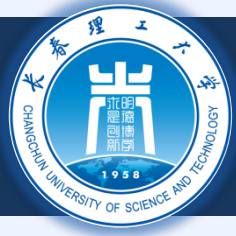
例

✦ 原型

- ✦ `A::operator char *() const;`
- ✦ 强制转换A类的对象到一个临时的 `char*` 对象
- ✦ `(char *)s` 这个强制转换实际上是一个函数调用:
`s.operator char*()`

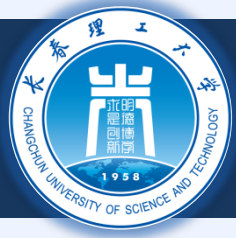
✦ 当然也可以是:

- ✦ `A::operator int() const;`
- ✦ `A::operator OtherClass() const;`



8.9 类型转换

- ◆ **强制类型转换可以避免一些重载的**
 - ✦ 假设String 可以强制类型转换为: `char *`
 - ✦ `cout << s; // s is a String`
 - ✦ **编译器隐式地将 s 转换为 `char *`**
 - ✦ **而不需要重载 `<<` 运算符**
 - ✦ 编译器只能做一次直接的类型强制转换
 - ✦ **不会进行间接的强制类型转换**



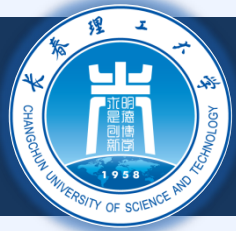
8.10 实例: String 类

◆ 构建自己的String类

- ✦ 实现字符串的创建和操作
- ✦ 在标准库中有string类

◆ 类型转换构造函数

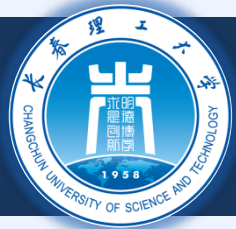
- ✦ 单个实参的构造函数
- ✦ 将别的类型的对象转换成当前的类对象
 - ✦ `String s1("hi");`
 - ✦ 从一个 `char *` 的对象创建一个String 的对象
- ✦ 任何一个单参数的构造函数都可以看作是一个类型转换



8.10 实例: String 类

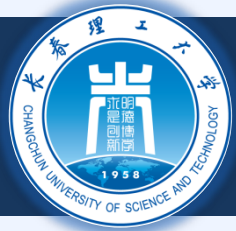
```
1 // string1.h
2 // String class definition.
3 #ifndef STRING1_H
4 #define STRING1_H
5
6 #include <iostream>
7
8 using std::ostream;
9 using std::istream;
10
11 class String {
12     friend ostream &operator<<( ostream &, const String & );
13     friend istream &operator>>( istream &, String & );
14
15 public:
16     String( const char * = "" ); // conversion/default ctor
17     String( const String & );    // copy constructor
18     ~String();                  // destructor
19
20     const String &operator=( const String & ); // assignment
21     const String &operator+=( const String & ); // concatenation
22
23     bool operator!() const; // is String empty?
24     bool operator==( const String & ) const; // test s1 == s2
25     bool operator<( const String & ) const; // test s1 < s2
```

支持: S+="abc"
为什么?



8.10 实例: String 类

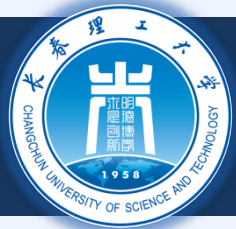
```
27 // test s1 != s2
28 bool operator!=( const String & right ) const
29 {
30     return !( *this == right );
31 } // end function operator!=
32
33
34 // test s1 > s2
35 bool operator>( const String &right ) const
36 {
37     return right < *this;
38 } // end function operator>
39
40
41 // test s1 <= s2
42 bool operator<=( const String &right ) const
43 {
44     return !( right < *this );
45 } // end function operator <=
46
47
48 // test s1 >= s2
49 bool operator>=( const String &right ) const
50 {
51     return !( *this < right );
52 }
53 } // end function operator>=
```



8.10 实例: String 类

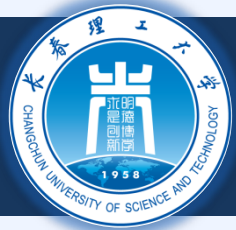
```
54
55     char &operator[] ( int );           // subscript operator
56     const char &operator[] ( int ) const; // subscript operator
57
58     String operator() ( int, int );      // return a substring
59
60     int getLength() const;              // return string length
61
62     private:
63         int length; // string length
64         char *sPtr; // pointer to start of string
65
66         void setString( const char * ); // utility function
67
68     }; // end class String
69
70 #endif
```

可以有任意的操作数



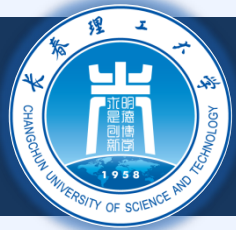
8.10 实例: String 类

```
1 // string1.cpp
2 // Member function definitions for class String.
3 #include <iostream>
4
5 using std::cout;
6 using std::endl;
7 #include <iomanip>
8
9
10 using std::setw;
11
12 #include <new> // C++ standard "new" operator
13
14 #include <cstring> // strcpy and strcat prototypes
15 #include <cstdlib> // exit prototype
16
17 #include "string1.h" // String class definition
18
19 // conversion constructor converts char * to String
20 String::String( const char *s )
21     : length( strlen( s ) )
22 {
23     cout << "Conversion constructor: " << s << '\n';
24     setString( s ); // call utility function
25
26 } // end String conversion constructor
```



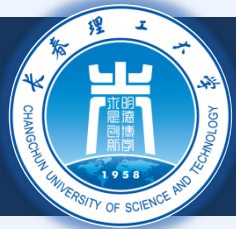
8.10 实例: String 类

```
28 // copy constructor
29 String::String( const String &copy )
30     : length( copy.length )
31 {
32     cout << "Copy constructor: " << copy.sPtr << '\n';
33     setString( copy.sPtr ); // call utility function
35 } // end String copy constructor
36
37 // destructor
38 String::~~String()
39 {
40     cout << "Destructor: " << sPtr << '\n';
41     delete [] sPtr;          // reclaim string
43 } // end ~String destructor
44
45 // overloaded = operator; avoids self assignment
46 const String &String::operator=( const String &right )
47 {
48     cout << "operator= called\n";
49     if ( &right != this ) {          // avoid self assignment
50         delete [] sPtr;              // prevents memory leak
51         length = right.length;       // new String length
52         setString( right.sPtr );     // call utility function
53     }
54 }
```



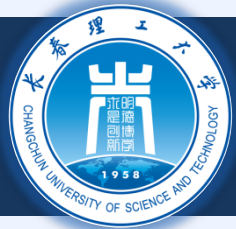
8.10 实例: String 类

```
56     else
57         cout << "Attempted assignment of a String to itself\n";
58
59     return *this;    // enables cascaded assignments
60
61 } // end function operator=
62
63 // concatenate right operand to this object and
64 // store in this object.
65 const String &String::operator+=( const String &right )
66 {
67     size_t newLength = length + right.length;    // new length
68     char *tempPtr = new char[ newLength + 1 ];    // create memory
69
70     strcpy( tempPtr, sPtr );                      // copy sPtr
71     strcpy( tempPtr + length, right.sPtr );       // copy right.sPtr
72
73     delete [] sPtr;                               // reclaim old space
74     sPtr = tempPtr;                               // assign new array to sPtr
75     length = newLength;                           // assign new length to length
76
77     return *this;    // enables cascaded calls
78
79 } // end function operator+=
```



8.10 实例: String 类

```
81 // is this String empty?
82 bool String::operator!() const
83 {
84     return length == 0;
85
86 } // end function operator!
87
88 // is this String equal to right String?
89 bool String::operator==( const String &right ) const
90 {
91     return strcmp( sPtr, right.sPtr ) == 0;
92
93 } // end function operator==
94
95 // is this String less than right String?
96 bool String::operator<( const String &right ) const
97 {
98     return strcmp( sPtr, right.sPtr ) < 0;
99
100 } // end function operator<
101
```



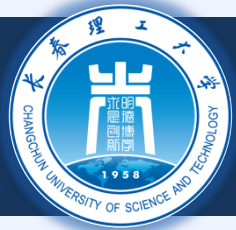
8.10 实例: String 类

```
102 // return reference to character in String as lvalue
103 char &String::operator[] ( int subscript )
104 {
105     // test for subscript out of range
106     if ( subscript < 0 || subscript >= length ) {
107         cout << "Error: Subscript " << subscript
108             << " out of range" << endl;
109         exit( 1 ); // terminate program
110     }
111     return sPtr[ subscript ]; // creates lvalue
112 }
113 // end function operator[]
114
115 // return reference to character in String as rvalue
116 const char &String::operator[] ( int subscript ) const
117 {
118     // test for subscript out of range
119     if ( subscript < 0 || subscript >= length ) {
120         cout << "Error: Subscript " << subscript
121             << " out of range" << endl;
122         exit( 1 ); // terminate program
123     }
124     return sPtr[ subscript ]; // creates rvalue
125 }
126 // end function operator[]
```



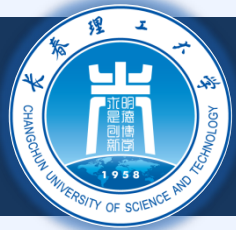
8.10 实例: String 类

```
132 // return a substring beginning at index and
133 // of length subLength
134 String String::operator()( int index, int subLength )
135 {
136     // if index is out of range or substring length < 0,
137     // return an empty String object
138     if ( index < 0 || index >= length || subLength < 0 )
139         return ""; // converted to a String object automatically
140
141     // determine length of substring
142     int len;
143
144     if ( ( subLength == 0 ) || ( index + subLength > length ) )
145         len = length - index;
146     else
147         len = subLength;
148
149     // allocate temporary array for substring and
150     // terminating null character
151     char *tempPtr = new char[ len + 1 ];
152
153     // copy substring into char array and terminate string
154     strncpy( tempPtr, &sPtr[ index ], len );
155     tempPtr[ len ] = '\\0';
```

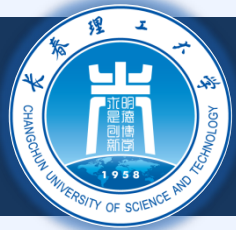
8.10 实例: String 类

```
156
157     // create temporary String object containing the substring
158     String tempString( tempPtr );
159     delete [] tempPtr; // delete temporary array
160
161     return tempString; // return copy of the temporary String
162
163 } // end function operator()
164
165 // return string length
166 int String::getLength() const
167 {
168     return length;
169
170 } // end function getLenth
171
172 // utility function called by constructors and operator=
173 void String::setString( const char *string2 )
174 {
175     sPtr = new char[ length + 1 ]; // allocate memory
176     strcpy( sPtr, string2 );      // copy literal to object
177
178 } // end function setString
```



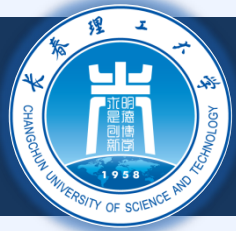
8.10 实例: String 类

```
179
180 // overloaded output operator
181 ostream &operator<<( ostream &output, const String &s )
182 {
183     output << s.sPtr;
184
185     return output;    // enables cascading
186
187 } // end function operator<<
188
189 // overloaded input operator
190 istream &operator>>( istream &input, String &s )
191 {
192     char temp[ 100 ];    // buffer to store input
193
194     input >> setw( 100 ) >> temp;
195     s = temp;            // use String class assignment operator
196
197     return input;        // enables cascading
198
199 } // end function operator>>
```



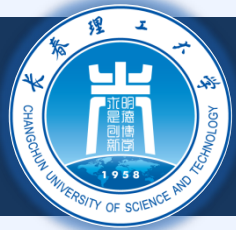
8.10 实例: String 类

```
2 // String class test program.
3 #include <iostream>
4
5 using std::cout;
6 using std::endl;
7
8 #include "string1.h"
9
10 int main()
11 {
12     String s1( "happy" );
13     String s2( " birthday" );
14     String s3;
15
16     // test overloaded equality and relational operators
17     cout << "s1 is \" << s1 << "\"; s2 is \" << s2
18         << "\"; s3 is \" << s3 << '\"'
19         << "\n\nThe results of comparing s2 and s1:"
20         << "\ns2 == s1 yields "
21         << ( s2 == s1 ? "true" : "false" )
22         << "\ns2 != s1 yields "
23         << ( s2 != s1 ? "true" : "false" )
24         << "\ns2 > s1 yields "
25         << ( s2 > s1 ? "true" : "false" )
```



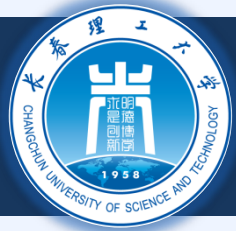
8.10 实例: String 类

```
26         << "\ns2 < s1 yields "
27         << ( s2 < s1 ? "true" : "false" )
28         << "\ns2 >= s1 yields "
29         << ( s2 >= s1 ? "true" : "false" )
30         << "\ns2 <= s1 yields "
31         << ( s2 <= s1 ? "true" : "false" );
32
33 // test overloaded String empty (!) operator
34 cout << "\n\nTesting !s3:\n";
35
36 if ( !s3 ) {
37     cout << "s3 is empty; assigning s1 to s3;\n";
38     s3 = s1; // test overloaded assignment
39     cout << "s3 is \"" << s3 << "\"";
40 }
41
42 // test overloaded String concatenation operator
43 cout << "\n\ns1 += s2 yields s1 = ";
44 s1 += s2; // test overloaded concatenation
45 cout << s1;
46
47 // test conversion constructor
48 cout << "\n\ns1 += \" to you\" yields\n";
49 s1 += " to you"; // test conversion constructor
50 cout << "s1 = " << s1 << "\n\n";
```



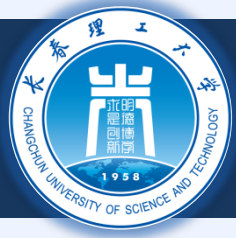
8.10 实例: String 类

```
51
52 // test overloaded function call operator () for substring
53 cout << "The substring of s1 starting at\n"
54      << "location 0 for 14 characters, s1(0, 14), is:\n"
55      << s1( 0, 14 ) << "\n\n";
56
57 // test substring "to-end-of-String" option
58 cout << "The substring of s1 starting at\n"
59      << "location 15, s1(15, 0), is: "
60      << s1( 15, 0 ) << "\n\n"; // 0 is "to end of string"
61
62 // test copy constructor
63 String *s4Ptr = new String( s1 );
64 cout << "\n*s4Ptr = " << *s4Ptr << "\n\n";
65
66 // test assignment (=) operator with self-assignment
67 cout << "assigning *s4Ptr to *s4Ptr\n";
68 *s4Ptr = *s4Ptr; // test overloaded assignment
69 cout << "*s4Ptr = " << *s4Ptr << '\n';
70
71 // test destructor
72 delete s4Ptr;
73
```



8.10 实例: String 类

```
74 // test using subscript operator to create lvalue
75 s1[ 0 ] = 'H';
76 s1[ 6 ] = 'B';
77 cout << "\ns1 after s1[0] = 'H' and s1[6] = 'B' is: "
78     << s1 << "\n\n";
79
80 // test subscript out of range
81 cout << "Attempt to assign 'd' to s1[30] yields:" << endl;
82 s1[ 30 ] = 'd';      // ERROR: subscript out of range
83
84 return 0;
85
86 } // end main
```



8.10 实例: String 类

Conversion constructor: happy

Conversion constructor: birthday

Conversion constructor:

s1 is "happy"; s2 is " birthday"; s3 is ""

The results of comparing s2 and s1:

s2 == s1 yields false

s2 != s1 yields true

s2 > s1 yields false

s2 < s1 yields true

s2 >= s1 yields false

s2 <= s1 yields true

Testing !s3:

s3 is empty; assigning s1 to s3;

operator= called

s3 is "happy"

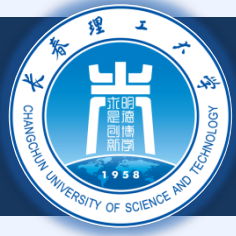
s1 += s2 yields s1 = happy birthday

s1 += " to you" yields

Conversion constructor: to you

Destructor: to you

s1 = happy birthday to you



8.10 实例: String 类

Conversion constructor: happy birthday

Copy constructor: happy birthday

Destructor: happy birthday

The substring of s1 starting at
location 0 for 14 characters, s1(0, 14), is:
happy birthday

Destructor: happy birthday

Conversion constructor: to you

Copy constructor: to you

Destructor: to you

The substring of s1 starting at
location 15, s1(15, 0), is: to you

Destructor: to you

Copy constructor: happy birthday to you

*s4Ptr = happy birthday to you

assigning *s4Ptr to *s4Ptr

operator= called

Attempted assignment of a String to itself

*s4Ptr = happy birthday to you

Destructor: happy birthday to you

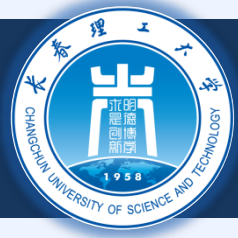


8.10 实例: String 类

`s1` after `s1[0] = 'H'` and `s1[6] = 'B'` is: Happy Birthday to you

Attempt to assign 'd' to `s1[30]` yields:

Error: Subscript 30 out of range



8.11 重载 ++ 和 --

◆ 自增/自减运算符可以被重载

- ✦ 将Date的一个对象d1的天数自增 1

- ✦ 原型 (成员函数)

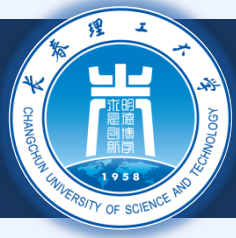
 - ✦ `Date &operator++();`

 - ✦ `++d1` 等同于 `d1.operator++()`

- ✦ 原型 (非成员函数)

 - ✦ `Friend Date &operator++( Date &);`

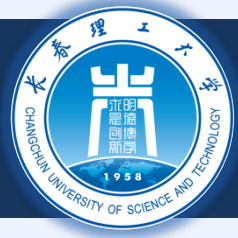
 - ✦ `++d1` 等同于 `operator++( d1 )`



8.11 重载 ++ 和 --

◆ 前置和后置自增的区别

- ✦ 为了区分前置和后置，后自增有一个空参数
 - ✦ 整数 0
- ✦ 原型 (成员函数)
 - ✦ `Date operator++( int );`
 - ✦ `d1++` 等同于 `d1.operator++( 0 )`
- ✦ 原型 (非成员函数)
 - ✦ `friend Date operator++( Data &, int );`
 - ✦ `d1++` 等同于 `operator++( d1, 0 )`
- ✦ 这个整型参数没有名字
 - ✦ 函数定义中也不出现



8.11 重载 ++ 和 --

返回

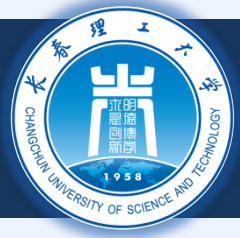
前自增

- 返回引用 (Date &)
- 左值 (可以被赋值)

后自增

- 返回值参
- 返回一个原有对象数值的临时对象
- 右值 (不可以放在赋值符的左侧)

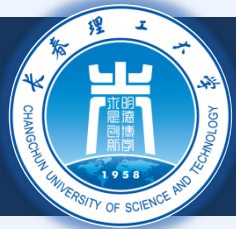
自减函数类似



8.12 实例: Date 类

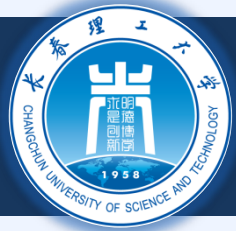
◆ 例: Date 类

- ✦ 重载自增运算符
 - ✦ 变更日、月和年
- ✦ 重载 += 运算符
- ✦ 测试闰年的功能
- ✦ 判断给定日期是否为当月最后一天的功能



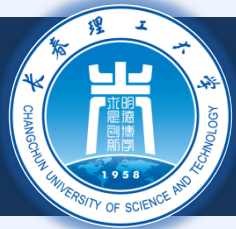
8.12 实例: Date 类

```
1 // date1.h
2 // Date class definition.
3 #ifndef DATE1_H
4 #define DATE1_H
5 #include <iostream>
6
7 using std::ostream;
8
9 class Date {
10     friend ostream &operator<<( ostream &, const Date & );
11
12 public:
13     Date( int m = 1, int d = 1, int y = 1900 ); // constructor
14     void setDate( int, int, int ); // set the date
15
16     Date &operator++(); // preincrement operator
17     Date operator++( int ); // postincrement operator
18
19     const Date &operator+=( int ); // add days, modify object
20
21     bool leapYear( int ) const; // is this a leap year?
22     bool endOfMonth( int ) const; // is this end of month?
```



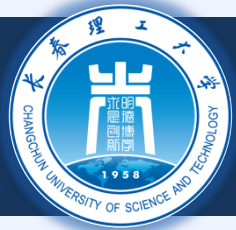
8.12 实例: Date 类

```
23
24     private:
25         int month;
26         int day;
27         int year;
28
29         static const int days[];           // array of days per month
30         void helpIncrement();              // utility function
31
32     }; // end class Date
33
34 #endif
```



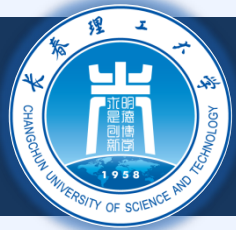
8.12 实例: Date 类

```
1 // date1.cpp
2 // Date class member function definitions.
3 #include <iostream>
4 #include "date1.h"
5
6 // initialize static member at file scope;
7 // one class-wide copy
8 const int Date::days[] =
9     { 0, 31, 28, 31, 30, 31, 30, 31, 31, 30, 31, 30, 31 };
10
11 // Date constructor
12 Date::Date( int m, int d, int y )
13 {
14     setDate( m, d, y );
15
16 } // end Date constructor
17
18 // set month, day and year
19 void Date::setDate( int mm, int dd, int yy )
20 {
21     month = ( mm >= 1 && mm <= 12 ) ? mm : 1;
22     year = ( yy >= 1900 && yy <= 2100 ) ? yy : 1900;
23 }
```

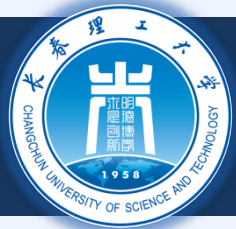
8.12 实例: Date 类

```
24 // test for a leap year
25 if ( month == 2 && leapYear( year ) )
26     day = ( dd >= 1 && dd <= 29 ) ? dd : 1;
27 else
28     day = ( dd >= 1 && dd <= days[ month ] ) ? dd : 1;
30 } // end function setDate
31
32 // overloaded preincrement operator
33 Date &Date::operator++()
34 {
35     helpIncrement();
37     return *this; // reference return to create an lvalue
39 } // end function operator++
40
41 // overloaded postincrement operator; note that the dummy
42 // integer parameter does not have a parameter name
43 Date Date::operator++( int )
44 {
45     Date temp = *this; // hold current state of object
46     helpIncrement();
47
48     // return unincremented, saved, temporary object
49     return temp; // value return; not a reference return
51 } // end function operator++
```



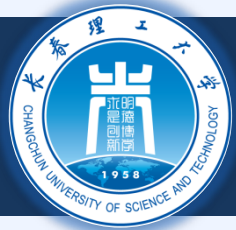
8.12 实例: Date 类

```
52
53 // add specified number of days to date
54 const Date &Date::operator+=( int additionalDays )
55 {
56     for ( int i = 0; i < additionalDays; i++ )
57         helpIncrement();
58
59     return *this;    // enables cascading
60
61 } // end function operator+=
62
63 // if the year is a leap year, return true;
64 // otherwise, return false
65 bool Date::leapYear( int testYear ) const
66 {
67     if ( testYear % 400 == 0 ||
68         ( testYear % 100 != 0 && testYear % 4 == 0 ) )
69         return true;    // a leap year
70     else
71         return false;    // not a leap year
72
73 } // end function leapYear
74
```



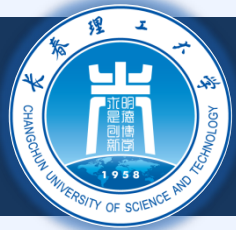
8.12 实例: Date 类

```
75 // determine whether the day is the last day of the month
76 bool Date::endOfMonth( int testDay ) const
77 {
78     if ( month == 2 && leapYear( year ) )
79         return testDay == 29; // last day of Feb. in leap year
80     else
81         return testDay == days[ month ];
82
83 } // end function endOfMonth
84
85 // function to help increment the date
86 void Date::helpIncrement()
87 {
88     // day is not end of month
89     if ( !endOfMonth( day ) )
90         ++day;
91
92     else
93
94         // day is end of month and month < 12
95         if ( month < 12 ) {
96             ++month;
97             day = 1;
98         }
```



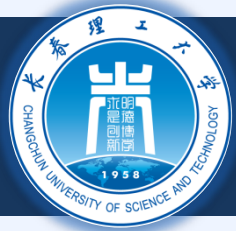
8.12 实例: Date 类

```
100         // last day of year
101     else {
102         ++year;
103         month = 1;
104         day = 1;
105     }
106
107 } // end function helpIncrement
108
109 // overloaded output operator
110 ostream &operator<<( ostream &output, const Date &d )
111 {
112     static char *monthName[ 13 ] = { "", "January",
113         "February", "March", "April", "May", "June",
114         "July", "August", "September", "October",
115         "November", "December" };
116
117     output << monthName[ d.month ] << ' '
118         << d.day << ", " << d.year;
119
120     return output;    // enables cascading
121
122 } // end function operator<<
```



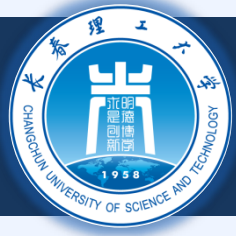
8.12 实例: Date 类

```
2 // Date class test program.
3 #include <iostream>
4
5 using std::cout;
6 using std::endl;
7
8 #include "date1.h" // Date class definition
9
10 int main()
11 {
12     Date d1; // defaults to January 1, 1900
13     Date d2( 12, 27, 1992 );
14     Date d3( 0, 99, 8045 ); // invalid date
15
16     cout << "d1 is " << d1 << "\nd2 is " << d2
17         << "\nd3 is " << d3;
18
19     cout << "\n\nd2 += 7 is " << ( d2 += 7 );
20
21     d3.setDate( 2, 28, 1992 );
22     cout << "\n\n d3 is " << d3;
23     cout << "\n++d3 is " << ++d3;
24
25     Date d4( 7, 13, 2002 );
```



8.12 实例: Date 类

```
26
27     cout << "\n\nTesting the preincrement operator:\n"
28         << "    d4 is " << d4 << '\n';
29     cout << "++d4 is " << ++d4 << '\n';
30     cout << "    d4 is " << d4;
31
32     cout << "\n\nTesting the postincrement operator:\n"
33         << "    d4 is " << d4 << '\n';
34     cout << "d4++ is " << d4++ << '\n';
35     cout << "    d4 is " << d4 << endl;
36
37     return 0;
38
39 } // end main
```



8.12 实例: Date 类

```
d1 is January 1, 1900
```

```
d2 is December 27, 1992
```

```
d3 is January 1, 1900
```

```
d2 += 7 is January 3, 1993
```

```
d3 is February 28, 1992
```

```
++d3 is February 29, 1992
```

```
Testing the preincrement operator:
```

```
d4 is July 13, 2002
```

```
++d4 is July 14, 2002
```

```
d4 is July 14, 2002
```

```
Testing the postincrement operator:
```

```
d4 is July 14, 2002
```

```
d4++ is July 14, 2002
```

```
d4 is July 15, 2002
```



8.13 标准库中的 string 和 vector 类

◆ C++中的内置类

- 每个人都可以使用
- string
 - 类似于前面的String类
- vector
 - 可以动态修改尺寸的数组

◆ 重写 String 和 Array 的例子

- 采用 string 和 vector



8.13 标准库中的 string 和 vector 类

◆ string 类

- 头文件<string>, namespace std
- 支持初始化 string s1("hi");
- 重载 <<
 - cout << s1
- 重载了关系运算符
 - == != >= > <= <
- 重载了赋值运算符 =
- 重载了连接符 (重载+=)



8.13 标准库中的 string 和 vector 类

◆ string 类

✦ 子串函数 substr

- ✦ s1.substr(0, 14);

- ➔ 从位置 0 开始, 返回 14 个字符

- ✦ S1.substr(15)

- ➔ 从位置15开始, 往后

✦ 重载 []

- ✦ 访问一个元素

- ✦ 没有范围检查 (if subscript invalid)

✦ at 函数

- ✦ s1.at(10)

- ✦ 返回下标10处的字符

- ✦ 有边界检车

- ➔ 如果无效会结束程序



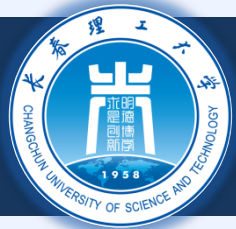
8.13 标准库中的 string 和 vector 类

```
2 // Standard library string class test program.
3 #include <iostream>
4
5 using std::cout;
6 using std::endl;
7
8 #include <string>
9
10 using std::string;
11
12 int main()
13 {
14     string s1( "happy" );
15     string s2( " birthday" );
16     string s3;
17
18     // test overloaded equality and relational operators
19     cout << "s1 is \" << s1 << "\"; s2 is \" << s2
20         << "\"; s3 is \" << s3 << '\"'
21         << "\n\nThe results of comparing s2 and s1:"
22         << "\ns2 == s1 yields "
23         << ( s2 == s1 ? "true" : "false" )
24         << "\ns2 != s1 yields "
25         << ( s2 != s1 ? "true" : "false" )
```



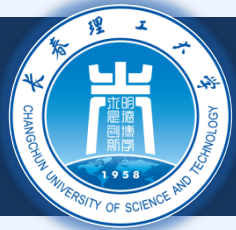
8.13 标准库中的 string 和 vector 类

```
26         << "\ns2 > s1 yields "
27         << ( s2 > s1 ? "true" : "false" )
28         << "\ns2 < s1 yields "
29         << ( s2 < s1 ? "true" : "false" )
30         << "\ns2 >= s1 yields "
31         << ( s2 >= s1 ? "true" : "false" )
32         << "\ns2 <= s1 yields "
33         << ( s2 <= s1 ? "true" : "false" );
34
35 // test string member function empty
36 cout << "\n\nTesting s3.empty():\n";
37
38 if ( s3.empty() ) {
39     cout << "s3 is empty; assigning s1 to s3;\n";
40     s3 = s1; // assign s1 to s3
41     cout << "s3 is \"" << s3 << "\"";
42 }
43
44 // test overloaded string concatenation operator
45 cout << "\n\ns1 += s2 yields s1 = ";
46 s1 += s2; // test overloaded concatenation
47 cout << s1;
48
```



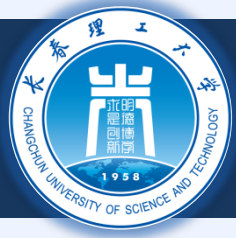
8.13 标准库中的 string 和 vector 类

```
49 // test overloaded string concatenation operator
50 // with C-style string
51 cout << "\n\s1 += \" to you\" yields\n";
52 s1 += " to you";
53 cout << "s1 = " << s1 << "\n\n";
54
55 // test string member function substr
56 cout << "The substring of s1 starting at location 0 for\n"
57      << "14 characters, s1.substr(0, 14), is:\n"
58      << s1.substr( 0, 14 ) << "\n\n";
59
60 // test substr "to-end-of-string" option
61 cout << "The substring of s1 starting at\n"
62      << "location 15, s1.substr(15), is:\n"
63      << s1.substr( 15 ) << '\n';
64
65 // test copy constructor
66 string *s4Ptr = new string( s1 );
67 cout << "\n*s4Ptr = " << *s4Ptr << "\n\n";
68
69 // test assignment (=) operator with self-assignment
70 cout << "assigning *s4Ptr to *s4Ptr\n";
71 *s4Ptr = *s4Ptr;
72 cout << "*s4Ptr = " << *s4Ptr << '\n';
```



8.13 标准库中的 string 和 vector 类

```
74 // test destructor
75 delete s4Ptr;
76
77 // test using subscript operator to create lvalue
78 s1[ 0 ] = 'H';
79 s1[ 6 ] = 'B';
80 cout << "\ns1 after s1[0] = 'H' and s1[6] = 'B' is: "
81      << s1 << "\n\n";
82
83 // test subscript out of range with string member function "at"
84 cout << "Attempt to assign 'd' to s1.at( 30 ) yields:" << endl;
85 s1.at( 30 ) = 'd';      // ERROR: subscript out of range
86
87 return 0;
88
89 } // end main
```



8.13 标准库中的 string 和 vector 类

```
s1 is "happy"; s2 is " birthday"; s3 is ""
```

The results of comparing s2 and s1:

```
s2 == s1 yields false
```

```
s2 != s1 yields true
```

```
s2 > s1 yields false
```

```
s2 < s1 yields true
```

```
s2 >= s1 yields false
```

```
s2 <= s1 yields true
```

Testing s3.empty() :

```
s3 is empty; assigning s1 to s3;
```

```
s3 is "happy"
```

```
s1 += s2 yields s1 = happy birthday
```

```
s1 += " to you" yields
```

```
s1 = happy birthday to you
```

The substring of s1 starting at location 0 for

14 characters, s1.substr(0, 14), is:

```
happy birthday
```



8.13 标准库中的 string 和 vector 类

The substring of s1 starting at
location 15, s1.substr(15), is:
to you

```
*s4Ptr = happy birthday to you
```

```
assigning *s4Ptr to *s4Ptr  
*s4Ptr = happy birthday to you
```

s1 after s1[0] = 'H' and s1[6] = 'B' is: Happy Birthday to you

Attempt to assign 'd' to s1.at(30) yields:

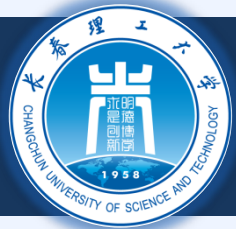
abnormal program termination



8.13 标准库中的 string 和 vector 类

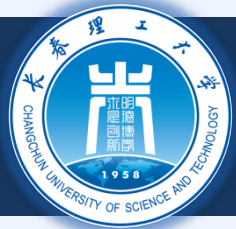
◆ vector 类

- ✦ 头文件 `<vector>`, namespace `std`
- ✦ 可以存储任何类型
 - ✦ `vector< int > myArray(10)`
- ✦ 函数 `size ( myArray.size() )`
- ✦ 重载 `[]`
 - ✦ 获得指定的元素, `myArray[3]`
- ✦ 重载 `!=`, `==`, and `=`
 - ✦ 不等, 相等, 赋值



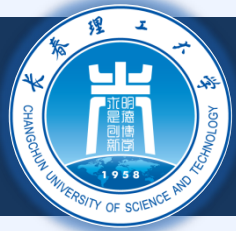
8.13 标准库中的 string 和 vector 类

```
2 // Demonstrating standard library class vector.
3 #include <iostream>
4
5 using std::cout;
6 using std::cin;
7 using std::endl;
8
9 #include <iomanip>
10
11 using std::setw;
12
13 #include <vector>
14
15 using std::vector;
16
17 void outputVector( const vector< int > & );
18 void inputVector( vector< int > & );
19
20 int main()
21 {
22     vector< int > integers1( 7 );    // 7-element vector< int >
23     vector< int > integers2( 10 );  // 10-element vector< int >
24 }
```



8.13 标准库中的 string 和 vector 类

```
25 // print integers1 size and contents
26 cout << "Size of vector integers1 is "
27     << integers1.size()
28     << "\nvector after initialization:\n";
29 outputVector( integers1 );
30
31 // print integers2 size and contents
32 cout << "\nSize of vector integers2 is "
33     << integers2.size()
34     << "\nvector after initialization:\n";
35 outputVector( integers2 );
36
37 // input and print integers1 and integers2
38 cout << "\nInput 17 integers:\n";
39 inputVector( integers1 );
40 inputVector( integers2 );
41
42 cout << "\nAfter input, the vectors contain:\n"
43     << "integers1:\n";
44 outputVector( integers1 );
45 cout << "integers2:\n";
46 outputVector( integers2 );
47
48 // use overloaded inequality (!=) operator
49 cout << "\nEvaluating: integers1 != integers2\n";
```



8.13 标准库中的 string 和 vector 类

```
51     if ( integers1 != integers2 )
52         cout << "integers1 and integers2 are not equal\n";
53
54     // create vector integers3 using integers1 as an
55     // initializer; print size and contents
56     vector< int > integers3( integers1 ); // copy constructor
57
58     cout << "\nSize of vector integers3 is "
59         << integers3.size()
60         << "\nvector after initialization:\n";
61     outputVector( integers3 );
62
63
64     // use overloaded assignment (=) operator
65     cout << "\nAssigning integers2 to integers1:\n";
66     integers1 = integers2;
67
68     cout << "integers1:\n";
69     outputVector( integers1 );
70     cout << "integers2:\n";
71     outputVector( integers1 );
72
```



8.13 标准库中的 string 和 vector 类

```
73 // use overloaded equality (==) operator
74 cout << "\nEvaluating: integers1 == integers2\n";
75
76 if ( integers1 == integers2 )
77     cout << "integers1 and integers2 are equal\n";
78
79 // use overloaded subscript operator to create rvalue
80 cout << "\nintegers1[5] is " << integers1[ 5 ];
81
82 // use overloaded subscript operator to create lvalue
83 cout << "\n\nAssigning 1000 to integers1[5]\n";
84 integers1[ 5 ] = 1000;
85 cout << "integers1:\n";
86 outputVector( integers1 );
87
88 // attempt to use out of range subscript
89 cout << "\nAttempt to assign 1000 to integers1.at( 15 )"
90     << endl;
91 integers1.at( 15 ) = 1000; // ERROR: out of range
92
93 return 0;
94
95 } // end main
96
```



8.13 标准库中的 string 和 vector 类

```
97 // output vector contents
98 void outputVector( const vector< int > &array )
99 {
100     for ( int i = 0; i < array.size(); i++ ) {
101         cout << setw( 12 ) << array[ i ];
102
103         if ( ( i + 1 ) % 4 == 0 ) // 4 numbers per row of output
104             cout << endl;
105
106     } // end for
107
108     if ( i % 4 != 0 )
109         cout << endl;
110
111 } // end function outputVector
112
113 // input vector contents
114 void inputVector( vector< int > &array )
115 {
116     for ( int i = 0; i < array.size(); i++ )
117         cin >> array[ i ];
118
119 } // end function inputVector
```



8.13 标准库中的 string 和 vector 类

Size of vector integers1 is 7

vector after initialization:

0	0	0	0
0	0	0	

Size of vector integers2 is 10

vector after initialization:

0	0	0	0
0	0	0	0
0	0		

Input 17 integers:

1 2 3 4 5 6 7 8 9 10 11 12 13 14 15 16 17

After input, the vectors contain:

integers1:

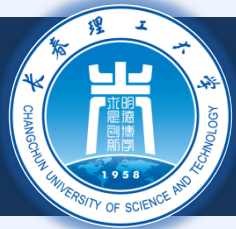
1	2	3	4
5	6	7	

integers2:

8	9	10	11
12	13	14	15
16	17		

Evaluating: integers1 != integers2

integers1 and integers2 are not equal



8.13 标准库中的 string 和 vector 类

Size of vector integers3 is 7

vector after initialization:

1	2	3	4
5	6	7	

Assigning integers2 to integers1:

integers1:

8	9	10	11
12	13	14	15
16	17		

integers2:

8	9	10	11
12	13	14	15
16	17		

Evaluating: integers1 == integers2

integers1 and integers2 are equal

integers1[5] is 13

Assigning 1000 to integers1[5]

integers1:

8	9	10	11
12	1000	14	15
16	17		

Attempt to assign 1000 to integers1.at(15)

abnormal program termination